

DBMS Group Term Project Proposal

Team Name : PreferNotToSay

Members:- Ashish Kumar(22CS30014),Gaurang Gupta(22CS30025),Shshank Mittal(22CS30051),Sujaan Jain(22CS10075)

Title:

Simulation and Analysis of Buffer Pool Manager Strategies for DBMS

Objective:

This project aims to simulate and analyze the performance of various buffer management strategies (LRU, MRU, CLOCK, and pinned blocks) in managing database pages for simple Join and Selection queries on small tables. The simulation will be implemented in C++ and integrated with the SQLite C library for realistic disk-oriented database operations. The project will examine and compare these buffer management strategies based on the efficiency measured by the number of disk I/O operations. To develop a clear understanding of buffer management in disk-oriented DBMS by practically implementing and analyzing popular replacement strategies, identifying their strengths and weaknesses in various scenarios.

Project Goals:

- Implement different buffer replacement strategies: Least Recently Used (LRU), Most Recently Used (MRU), CLOCK, and management of pinned blocks.
- Integrate SQLite C library for simulating realistic database interactions.
- Perform comparative analysis by measuring disk I/O required for join/selection queries.

Methodology:

1. **Implementation of Buffer Pool:**
 - Construct a buffer pool managing fixed-size pages (4 KB each) and simulate frame management.
 - Ensure thread safety using mutexes (latches) to handle concurrent access.
2. **Replacement Policies Implementation:**
 - Implement buffer replacement strategies:
 - Least Recently Used (LRU)
 - Most Recently Used (MRU)
 - CLOCK
 - Pinned Blocks (non-evictable)
3. **Integration with SQLite:**
 - Utilize SQLite's C library for query processing.
4. **Performance Evaluation:**
 - Develop test cases involving basic JOIN and Selection queries.
 - Measure and record disk I/O for each strategy under various workloads.
5. **Comparative Analysis:**

- Analyze performance metrics, specifically focusing on the total disk I/O operations.
- Compare LRU, MRU, CLOCK, and pinned-block strategies to determine their efficiency.

Expected Outcomes:

- Clear insights into buffer management performance.
- Identification of optimal scenarios for each replacement strategy.
- Practical knowledge of database system internals and performance tuning.

Deliverables:

- Fully implemented simulation code.
- Documentation and intermediate progress reports.
- Final comprehensive report with implementation details, methodology, results (including screenshots and comparative analysis), and conclusions.

Abstract:

- We will implement the following classes in C++:

LRUreplacer: maintain timestamp for each frame and evict least recently used frame

MRUreplacer: maintain frame_id of the most recently used page and evict this when required

CLOCKreplacer: maintain reference_bit for each frame and a common pointer which points to the packet to be evicted next.

All these classes will have the facility of pinning pages(a page will be considered for eviction if its pin_count is 0)

- The above classes help in maintaining which all pages are present in the buffer pool.After this we need other classes which actually perform the read/write operation on the disk if required.For this we implement following 2 classes:

DiskScheduler: maintains a queue of disk operations(represented by diskRequest) which are to be performed on the disk (as requested by possibly different threads) and forwards each request to the next class,DiskManager.

DiskManager: Its function is to read/write from/to disk(for simulation it only maintains the count of read/write operations) (DiskScheduler uses this class)

- Finally, class BufferPoolManager using DiskScheduler and LRU/MRU/CLOCKreplacer
- Now, integrating BufferPoolManager with VFS(virtual file system) of SQLite helps us get the all the queried addresses by the QueryProcessor of SQLite and instead of doing

read() and write() provided by OS we instead do read(),write() of DiskManager class implemented above by defining them in custom VFS.

Timeline:

- March 10-24, 2024 : **Implementation of classes LRUreplacer,MRUreplacer,CLOCKreplacer,DiskScheduler,DiskManager,BufferPool Manager**
- March 25-April 7, 2024 : **Integrating with SQLite. Using custom VFS(virtual file system) definition so that we can get count of all the different types of operations(diskRead,diskWrite).**
- April 8-10, 2024: **Comparing performance of different strategies and with inbuilt SQLite bufferManager.**
- April 15, 2024: **Final Report Submission & Demonstration**

References:

- SQLite Documentation: <https://sqlite.org/index.html>
- CMU 15-445/645 Project Reference (BusTub - Buffer Pool Manager)